

Chapter 12. New Date and Time API

This chapter covers

- Why we needed a new date and time library, introduced in Java 8
- Representing date and time for both humans and machines
- Defining an amount of time
- Manipulating, formatting, and parsing dates
- Dealing with different time zones and calendars

The Java API includes many useful components to help you build complex applications. Unfortunately, the Java API isn't always perfect. We believe that the majority of experienced Java developers will agree that date and time support before Java 8 was far from ideal. Don't worry, though; Java 8 introduces a brand-new Date and Time API to tackle this issue.

In Java 1.0, the only support for date and time was the `java.util.Date` class. Despite its name, this class doesn't represent a date, but a point in time with millisecond precision. Even worse, the usability of this class is harmed by some nebulous design decisions such as the choice of its off-sets: the years start from 1900, whereas the months start at index 0. If you wanted to represent the release date of Java 9, which is 21 September 2017, you'd have to create an instance of `Date` as follows:

```
Date date = new Date(117, 8, 21);
```

Printing this date produces, for the authors:

```
Thu Sep 21 00:00:00 CET 2017
```

Not very intuitive, is it? Moreover, even the `String` returned by the `toString` method of the `Date` class could be quite misleading. It also in-

Other formats available

 Audiobook 



one, CET, which is Central Europe Time in itself merely inserts the JVM default time

The problems and limitations of the `Date` class were immediately clear when Java 1.0 came out, but it was also clear that the problems couldn't be fixed without breaking its backward compatibility. As a consequence, in Java 1.1 many methods of the `Date` class were deprecated, and the

class was replaced by the alternative `java.util.Calendar` class. Unfortunately, `Calendar` has similar problems and design flaws that lead to error-prone code. Months also start at index 0. (At least `Calendar` got rid of the 1900 offset for the year.) Worse, the presence of *both* the `Date` and `Calendar` classes increases confusion among developers. (Which one should you use?) In addition, features such as `DateFormat`, used to format and parse dates or time in a language-independent manner, work only with the `Date` class.

The `DateFormat` comes with its own set of problems. It isn't thread-safe, for example, which means that if two threads try to parse a date by using the same formatter at the same time, you may receive unpredictable results.

Finally, both `Date` and `Calendar` are mutable classes. What does it mean to mutate the 21st of September 2017 to the 25th of October? This design choice can lead you into a maintenance nightmare, as you'll learn in more detail in [chapter 18](#), which is about functional programming.

The consequence is that all these flaws and inconsistencies have encouraged the use of third-party date and time libraries, such as Joda-Time. For these reasons, Oracle decided to provide high-quality date and time support in the native Java API. As a result, Java 8 integrates many of the Joda-Time features in the `java.time` package.

In this chapter, we explore the features introduced by the new Date and Time API. We start with basic use cases such as creating dates and times that are suitable to be used by both humans and machines. Then we gradually explore more-advanced applications of the new Date and Time API, such as manipulating, parsing, and printing date-time objects, and working with different time zones and alternative calendars.

12.1. `LocalDate`, `LocalTime`, `LocalDateTime`, `Instant`, `Duration`, and `Period`

We start by exploring how to create simple dates and intervals. The `java.time` package includes many new classes to help you: `LocalDate`, `LocalTime`, `LocalDateTime`, `Instant`, `Duration`, and `Period`.

Other formats available



✕ `Date` and `LocalTime`

`Date` is the first one you'll come across when you start using the new Date and Time API. An instance of this class is an immutable object representing a plain date without the time of day. In particular, it doesn't carry any information about the time zone.

You can create a `LocalDate` instance by using the `of` static factory method. A `LocalDate` instance provides many methods to read its most commonly used values (year, month, day of the week, and so on), as shown in the following listing.

Listing 12.1. Creating a `LocalDate` and reading its values

```

    LocalDate date = LocalDate.of(2017, 9, 21);    1
    int year = date.getYear();                    2
    Month month = date.getMonth();                3
    int day = date.getDayOfMonth();               4
    DayOfWeek dow = date.getDayOfWeek();          5
    int len = date.lengthOfMonth();               6
    boolean leap = date.isLeapYear();             7
```

- **1 2017-09-21**
- **2 2017**
- **3 SEPTEMBER**
- **4 21**
- **5 THURSDAY**
- **6 30 (days in September)**
- **7 false (not a leap year)**

It's also possible to obtain the current date from the system clock by using the `now` factory method:

```

    LocalDate today = LocalDate.now();
```

All the other date-time classes that we investigate in the remaining part of this chapter provide a similar factory method. You can also access the same information by passing a `TemporalField` to the `get` method. The `TemporalField` is an interface defining how to access the value of a specific field of a temporal object. The `ChronoField` enumeration implements this interface, so you can conveniently use an element of that enumeration with the `get` method, as shown in the next listing.

Listing 12.2. Reading `LocalDate` values by using a `TemporalField`

Other formats available



Audiobook



```

    ChronoField.YEAR);
    ChronoField.MONTH_OF_YEAR);
    ChronoField.DAY_OF_MONTH);
```

You could use the built-in `getYear()`, `getMonthValue()`, and `getDayOfMonth()` methods in a more-readable form to access the information as follows:

```
int year = date.getYear();
int month = date.getMonthValue();
int day = date.getDayOfMonth();
```

Similarly, the time of day, such as 13:45:20, is represented by the `LocalTime` class. You can create instances of `LocalTime` by using two overloaded static factory methods named `of`. The first one accepts an hour and a minute, and the second one also accepts a second. Like the `LocalDate` class, the `LocalTime` class provides some getter methods to access its values, as shown in the following listing.

Listing 12.3. Creating a `LocalTime` and reading its values

```
LocalTime time = LocalTime.of(13, 45, 20);    1
int hour = time.getHour();                  2
int minute = time.getMinute();              3
int second = time.getSecond();              4
```

- 1 13:45:20
- 2 13
- 3 45
- 4 20

You can create both `LocalDate` and `LocalTime` by parsing a `String` representing them. To achieve this task, use their `parse` static methods:

```
LocalDate date = LocalDate.parse("2017-09-21");
LocalTime time = LocalTime.parse("13:45:20");
```

It's possible to pass a `DateTimeFormatter` to the `parse` method. An instance of this class specifies how to format a date and/or a time object. It's intended to be a replacement for the old `java.util.DateFormat` that we mentioned earlier. We show in more detail how you can use a `DateTimeFormatter` in [section 12.2.2](#). Also note that both these `parse` methods throw a `DateTimeParseException`, which extends `RuntimeException` in case the `String` argument can't be parsed as a valid `LocalDate` or `LocalTime`.

Other formats available



Audiobook

a time

The composite class called `LocalDateTime` pairs a `LocalDate` and a `LocalTime`. It represents both a date and a time without a time zone and can be created directly or by combining a date and time, as shown in the following listing.

Listing 12.4. Creating a `LocalDateTime` directly or by combining a date and a time

```
// 2017-09-21T13:45:20
LocalDateTime dt1 = LocalDateTime.of(2017, Month.SEPTEMBER, 21, 13, 45);
LocalDateTime dt2 = LocalDateTime.of(date, time);
LocalDateTime dt3 = date.atTime(13, 45, 20);
LocalDateTime dt4 = date.atTime(time);
LocalDateTime dt5 = time.atDate(date);
```

Note that it's possible to create a `LocalDateTime` by passing a time to a `LocalDate` or a date to a `LocalTime`, using their `atTime` or `atDate` methods, respectively. You can also extract the `LocalDate` or `LocalTime` component from a `LocalDateTime` by using the `toLocalDate` and `toLocalTime` methods:

```
LocalDate date1 = dt1.toLocalDate();      1
LocalTime time1 = dt1.toLocalTime();      2
```

- 1 2017-09-21
- 2 13:45:20

12.1.3. Instant: a date and time for machines

As humans, we're used to thinking of dates and time in terms of weeks, days, hours, and minutes. Nonetheless, this representation isn't easy for a computer to deal with. From a machine point of view, the most natural format to model time is a single large number representing a point on a continuous timeline. This approach is used by the new `java.time.Instant` class, which represents the number of seconds passed since the Unix epoch time, set by convention to midnight of January 1, 1970 UTC.

You can create an instance of this class by passing the number of seconds to its `ofEpochSecond` static factory method. In addition, the `Instant` class supports nanosecond precision. A supplementary overloaded version of the `ofEpochSecond` static factory method accepts a second argument that's a nanosecond adjustment to the passed number of seconds. This overloaded version adjusts the nanosecond argument, ensuring that

Other formats available



Audiobook



is between 0 and 999,999,999. As a result, the `ofEpochSecond` factory method return

```
Instant.ofEpochSecond(3);
Instant.ofEpochSecond(3, 0);
```

```
Instant.ofEpochSecond(2, 1_000_000_000);      1
Instant.ofEpochSecond(4, -1_000_000_000);     2
```

- **1 One billion nanoseconds (1 second) after 2 seconds**
- **2 One billion nanoseconds (1 second) before 4 seconds**

As you’ve already seen for `LocalDate` and the other human-readable date-time classes, the `Instant` class supports another static factory method named `now`, which allows you to capture a timestamp of the current moment. It’s important to reinforce that an `Instant` is intended for use only by a machine. It consists of a number of seconds and nanoseconds. As a consequence, it doesn’t provide any ability to handle units of time that are meaningful to humans. A statement like

```
int day = Instant.now().get(ChronoField.DAY_OF_MONTH);
```

throws an exception like this:

```
java.time.temporal.UnsupportedTemporalTypeException: Unsupported field
    DayOfMonth
```

But you can work with `Instants` by using the `Duration` and `Period` classes, which we look at next.

12.1.4. Defining a Duration or a Period

All the classes you’ve seen so far implement the `Temporal` interface, which defines how to read and manipulate the values of an object modeling a generic point in time. We’ve shown you a few ways to create different `Temporal` instances. The next natural step is creating a duration between two temporal objects. The `between` static factory method of the `Duration` class serves exactly this purpose. You can create a duration between two `LocalTimes`, two `LocalDateTimes`, or two `Instants` as follows:

```
Duration d1 = Duration.between(time1, time2);
Duration d1 = Duration.between(dateTime1, dateTime2);
Duration d1 = Duration.between(instant1, instant2);
```

Other formats available



`Instant` are made for different purposes, one to be used by humans and the other by machines, you’re not allowed to mix them. If you try to create a duration between them, you’ll only obtain a `DateTimeException`. Moreover, because the `Duration` class is

used to represent an amount of time measured in seconds and eventually nanoseconds, you can't pass a `LocalDate` to the `between` method.

When you need to model an amount of time in terms of years, months, and days, you can use the `Period` class. You can find out the difference between two `LocalDates` with the `between` factory method of that class:

```
Period tenDays = Period.between(LocalDate.of(2017, 9, 11),
                                     LocalDate.of(2017, 9, 21));
```

Finally, the `Duration` and `Period` classes have other convenient factory methods to create instances of them directly, without defining them as the difference between two temporal objects, as shown in the following listing.

Listing 12.5. Creating Durations and Periods

```
Duration threeMinutes = Duration.ofMinutes(3);
Duration threeMinutes = Duration.of(3, ChronoUnit.MINUTES);
Period tenDays = Period.ofDays(10);
Period threeWeeks = Period.ofWeeks(3);
Period twoYearsSixMonthsOneDay = Period.of(2, 6, 1);
```

The `Duration` and `Period` classes share many similar methods, which [table 12.1](#) lists.

Table 12.1. The common methods of date-time classes representing an interval

Method	Static	Description
<code>between</code>	Yes	Creates an interval between two points in time
<code>from</code>	Yes	Creates an interval from a temporal unit
<code>parse</code>	Yes	Creates an instance of this interval from its constituent parts
<code>parse</code>	Yes	Creates an instance of this interval from a String

Other formats available

Audiobook

Method	Static	Description
<code>addTo</code>	No	Creates a copy of this interval, adding to it the specified temporal object
<code>get</code>	No	Reads part of the state of this interval
<code>isNegative</code>	No	Checks whether this interval is negative, excluding zero
<code>isZero</code>	No	Checks whether this interval is zero-length
<code>minus</code>	No	Creates a copy of this interval with an amount of time subtracted
<code>multipliedBy</code>	No	Creates a copy of this interval multiplied by the given scalar
<code>negated</code>	No	Creates a copy of this interval with the length negated
<code>plus</code>	No	Creates a copy of this interval with an amount of time added
<code>subtractFrom</code>	No	Subtracts this interval from the specified temporal object

All the classes we’ve investigated so far are immutable, which is a great design choice to allow a more functional programming style, ensure thread safety, and preserve the consistency of the domain model. Nevertheless, the new Date and Time API offers some handy methods for creating modified versions of those objects. You may want to add three days to an existing `LocalDate` instance, for example, and we explore how to do this in the next section. In addition, we explore how to create a date-time formatter from a given pattern, such as `dd/MM/yyyy`, or even programmatically, as well as how to use this formatter for both parsing

Other formats available

 Audiobook 



ing, and formatting dates

The most immediate and easiest way to create a modified version of an existing `LocalDate` is to change one of its attributes, using one of its `withAttribute` methods. Note that all the methods return a new object

with the modified attribute, as shown in [listing 12.6](#); they don't mutate the existing object!

Listing 12.6. Manipulating the attributes of a `LocalDate` in an absolute way

```
    LocalDate date1 = LocalDate.of(2017, 9, 21);           1
    LocalDate date2 = date1.withYear(2011);               2
    LocalDate date3 = date2.withDayOfMonth(25);           3
    LocalDate date4 = date3.with(ChronoField.MONTH_OF_YEAR, 2); 4
```

- 1 2017-09-21
- 2 2011-09-21
- 3 2011-09-25
- 4 2011-02-25

You can do the same thing with the more generic `with` method, taking a `TemporalField` as the first argument, as in the last statement of [listing 12.6](#). This last `with` method is the dual of the `get` method used in [listing 12.2](#). Both of these methods are declared in the `Temporal` interface implemented by all the classes, such as `LocalDate`, `LocalTime`, `LocalDateTime`, and `Instant`, of the Date and Time API. More precisely, the `get` and `with` methods let you respectively read and modify¹ fields of a `Temporal` object. They throw an `UnsupportedTemporalTypeException` if the requested field isn't supported by the specific `Temporal`, such as a `ChronoField.MONTH_OF_YEAR` on an `Instant` or a `ChronoField.NANO_OF_SECOND` on a `LocalDate`.

¹

Remember that such 'with' methods don't modify the existing Temporal object but create a copy with the specific field updated. This process is called a functional update (see [chapter 19](#)).

It's even possible to manipulate a `LocalDate` in a declarative manner. You can add or subtract a given amount of time, for example, as shown in [listing 12.7](#).

Other formats available



Manipulating the attributes of a `LocalDate` in a relative way

```
    LocalDate date1 = LocalDate.of(2017, 9, 21);           1
    LocalDate date2 = date1.plusWeeks(1);                 2
    LocalDate date3 = date2.minusYears(6);                 3
    LocalDate date4 = date3.plus(6, ChronoUnit.MONTHS);    4
```

- 1 2017-09-21
- 2 2017-09-28
- 3 2011-09-28
- 4 2012-03-28

Similarly to what we've explained about the `with` and `get` methods, the generic `plus` method used in the last statement of [listing 12.7](#), together with the analogous `minus` method, is declared in the `Temporal` interface. These methods allow you to move a `Temporal` back or forward a given amount of time, defined by a number plus a `TemporalUnit`, where the `ChronoUnit` enumeration offers a convenient implementation of the `TemporalUnit` interface.

As you may have anticipated, all the date-time classes representing a point in time such as `LocalDate`, `LocalTime`, `LocalDateTime`, and `Instant` have many methods in common. [Table 12.2](#) summarizes these methods.

Table 12.2. The common methods of date-time classes representing a point in time

Method	Static	Description
<code>from</code>	Yes	Creates an instance of this class from the passed temporal object
<code>now</code>	Yes	Creates a temporal object from the system clock
<code>of</code>	Yes	Creates an instance of this temporal object from its constituent parts
<code>parse</code>	Yes	Creates an instance of this temporal object from a <code>String</code>
<code>atOffset</code>	No	Combines this temporal object with a zone offset
<code>format</code>	No	Combines this temporal object with a time zone
		Converts this temporal object to a <code>String</code> by using the specified formatter (not available for <code>Instant</code>)

Other formats available



Audiobook



Method	Static	Description
<code>get</code>	No	Reads part of the state of this temporal object
<code>minus</code>	No	Creates a copy of this temporal object with an amount of time subtracted
<code>plus</code>	No	Creates a copy of this temporal object with an amount of time added
<code>with</code>	No	Creates a copy of this temporal object with part of the state changed

Check what you’ve learned up to now about manipulating dates with quiz 12.1.

Quiz 12.1: Manipulating a `LocalDate`

What will the value of the date variable be after the following manipulations?

```

    LocalDate date = LocalDate.of(2014, 3, 18);
    date = date.with(ChronoField.MONTH_OF_YEAR, 9);
    date = date.plusYears(2).minusDays(10);
    date.withYear(2011);

```

Answer:

```
2016-09-08
```

As you’ve seen, you can manipulate the date both in an absolute way and in a relative way. You can also concatenate more manipulations in a single statement, because every change creates a new `LocalDate` object, and the subsequent invocation manipulates the object created by the former one. Finally, the last statement in this code snippet has no observable

Other formats available



Audiobook



12.2.1. Working with `TemporalAdjusters`

All the date manipulations you’ve seen so far are relatively straightforward. Sometimes, though, you need to perform advanced operations, such as adjusting a date to the next Sunday, the next working day, or the last day of the month. In such cases, you can pass to an overloaded version of the `with` method a `TemporalAdjuster` that provides a more customizable way to define the manipulation needed to operate on a specific date. The Date and Time API already provides many predefined `TemporalAdjusters` for the most common use cases. You can access them by using the static factory methods contained in the `TemporalAdjusters` class, as shown in [listing 12.8](#).

Listing 12.8. Using the predefined `TemporalAdjusters`

```
import static java.time.temporal.TemporalAdjusters.*;
LocalDate date1 = LocalDate.of(2014, 3, 18);
LocalDate date2 = date1.with(nextOrSame(DayOfWeek.SUNDAY));
LocalDate date3 = date2.with(lastDayOfMonth());
```

- 1 2014-03-18
- 2 2014-03-23
- 3 2014-03-31

[Table 12.3](#) lists the `TemporalAdjusters` that you can create with these factory methods.

Table 12.3. The factory methods of the `TemporalAdjusters` class

Method	Description
<code>dayOfWeekInMonth</code>	Creates a new date in the same month with the ordinal day of week. (Negative numbers count backward from the end of the month.)
<code>firstDayOfMonth</code>	Creates a new date set to the first day of the current month.
<code>firstDayOfNextMonth</code>	Creates a new date set to the first day of the next month.
<code>firstDayOfNextYear</code>	Creates a new date set to the first day of the next year.
<code>firstDayOfYear</code>	Creates a new date set to the first day of the current year.

Other formats available

 Audiobook 

Method	Description
<code>firstInMonth</code>	Creates a new date in the same month with the first matching day of the week.
<code>lastDayOfMonth</code>	Creates a new date set to the last day of the current month.
<code>lastDayOfNextMonth</code>	Creates a new date set to the last day of the next month.
<code>lastDayOfNextYear</code>	Creates a new date set to the last day of the next year.
<code>lastDayOfYear</code>	Creates a new date set to the last day of the current year.
<code>lastInMonth</code>	Creates a new date in the same month with the last matching day of the week.
<code>next</code> <code>previous</code>	Creates a new date set to the first occurrence of the specified day of week after/before the date being adjusted.
<code>nextOrSame</code> <code>previousOrSame</code>	Creates a new date set to the first occurrence of the specified day of week after/before the date being adjusted unless it's already on that day, in which case the same object is returned.

As you can see, `TemporalAdjusters` allow you to perform more-complex date manipulations that still read like the problem statement. Moreover, it's relatively simple to create your own custom `TemporalAdjuster` implementation if you can't find a predefined `TemporalAdjuster` that fits your needs. In fact, the `TemporalAdjuster` interface declares only a single method (which makes it a functional interface), defined as shown in the following listing.

Other formats available



`TemporalAdjuster` interface

```
public interface TemporalAdjuster {  
    Temporal adjustInto(Temporal temporal);  
}
```

This example means that an implementation of the `TemporalAdjuster` interface defines how to convert a `Temporal` object to another `Temporal`. You can think of a `TemporalAdjuster` as being like a `UnaryOperator<Temporal>`. Take a few minutes to practice what you've learned so far and implement your own `TemporalAdjuster` in quiz 12.2.

Quiz 12.2: Implementing a custom `TemporalAdjuster`

Develop a class named `NextWorkingDay`, implementing the `TemporalAdjuster` interface that moves a date forward by one day but skips Saturdays and Sundays. Using

```
date = date.with(new NextWorkingDay());
```

should move the date to the next day, if this day is between Monday and Friday, but to the next Monday if it's a Saturday or a Sunday.

Answer:

You can implement the `NextWorkingDay` adjuster as follows:

```
public class NextWorkingDay implements TemporalAdjuster {
    @Override
    public Temporal adjustInto(Temporal temporal) {
        DayOfWeek dow =
            DayOfWeek.of(temporal.get(ChronoField.DAY_OF_WEEK));
        int dayToAdd = 1;
        if (dow == DayOfWeek.FRIDAY) dayToAdd = 3;
        else if (dow == DayOfWeek.SATURDAY) dayToAdd = 2;
        return temporal.plus(dayToAdd, ChronoUnit.DAYS);
    }
}
```

- **1 Read the current day.**
- **2 Normally add one day.**
- **3 But add three days if today is a Friday.**
- **4 Add two days if today is a Saturday.**
- **5 Return the modified date adding the right number of days.**

Other formats available



...ally moves a date forward one day, except ...day, in which case it advances the dates by three or two days, respectively. Note that because a `TemporalAdjuster` is a functional interface, you could pass the behavior of this adjuster in a lambda expression:

```

date = date.with(temporal -> {
    DayOfWeek dow =
        DayOfWeek.of(temporal.get(ChronoField.DAY_OF_WEEK));
    int dayToAdd = 1;
    if (dow == DayOfWeek.FRIDAY) dayToAdd = 3;
    else if (dow == DayOfWeek.SATURDAY) dayToAdd = 2;
    return temporal.plus(dayToAdd, ChronoUnit.DAYS);
});

```

You're likely to want to apply this manipulation to a date in several points of your code, and for this reason, we suggest encapsulating its logic in a proper class, as we did here. Do the same for all the manipulations you use frequently. You'll end up with a small library of adjusters that you and your team can easily reuse in your codebase.

If you want to define the `TemporalAdjuster` with a lambda expression, it's preferable to do so by using the `ofDateAdjuster` static factory of the `TemporalAdjusters` class, which accepts a `UnaryOperator<LocalDate>` as follows:

```

TemporalAdjuster nextWorkingDay = TemporalAdjusters.ofDateAdjuster(
    temporal -> {
        DayOfWeek dow =
            DayOfWeek.of(temporal.get(ChronoField.DAY_OF_WEEK));
        int dayToAdd = 1;
        if (dow == DayOfWeek.FRIDAY) dayToAdd = 3;
        else if (dow == DayOfWeek.SATURDAY) dayToAdd = 2;
        return temporal.plus(dayToAdd, ChronoUnit.DAYS);
    });
date = date.with(nextWorkingDay);

```

Another common operation that you may want to perform on your date and time objects is printing them in different formats specific to your business domain. Similarly, you may want to convert `Strings` representing dates in those formats to actual date objects. In the next section, we demonstrate the mechanisms provided by the new Date and Time API to accomplish these tasks.

Other formats available



date-time objects

Other relevant features for working with dates and times. The new `java.time.format` package is devoted to these purposes. The most important class of this package is `DateTimeFormatter`. The easiest way to create a formatter is through its static factory methods and constants. The constants such as

BASIC_ISO_DATE and ISO_LOCAL_DATE are predefined instances of the DateTimeFormatter class. You can use all DateTimeFormatters to create a String representing a given date or time in a specific format. Here, for example, we produce a String by using two different formatters:

```
LocalDate date = LocalDate.of(2014, 3, 18);  
String s1 = date.format(DateTimeFormatter.BASIC_ISO_DATE);      1  
String s2 = date.format(DateTimeFormatter.ISO_LOCAL_DATE);      2
```

- 1 20140318
- 2 2014-03-18

You can also parse a String representing a date or a time in that format to re-create the date object itself. You can achieve this task by using the parse factory method provided by all the classes of the Date and Time API representing a point in time or an interval:

```
LocalDate date1 = LocalDate.parse("20140318",  
                                   DateTimeFormatter.BASIC_ISO_DATE);  
LocalDate date2 = LocalDate.parse("2014-03-18",  
                                   DateTimeFormatter.ISO_LOCAL_DATE);
```

In comparison with the old `java.util.DateFormat` class, all the `DateTimeFormatter` instances are thread-safe. Therefore, you can create singleton formatters like the ones defined by the `DateTimeFormatter` constants and share them among multiple threads. The next listing shows how the `DateTimeFormatter` class also supports a static factory method that lets you create a formatter from a specific pattern.

Listing 12.10. Creating a `DateTimeFormatter` from a pattern

```
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy")  
LocalDate date1 = LocalDate.of(2014, 3, 18);  
String formattedDate = date1.format(formatter);  
LocalDate date2 = LocalDate.parse(formattedDate, formatter);
```

Here, the `LocalDate`'s `format` method produces a String representing

Other formats available



pattern. Next, the static `parse` method re-creates the generated String, using the same formatter. `DateTimeFormatter` also has an overloaded version that allows you to create a formatter for a given Locale, as shown in the following listing.

Listing 12.11. Creating a localized `DateTimeFormatter`


```

DateTimeFormatter italianFormatter =
    DateTimeFormatter.ofPattern("d. MMMM yyyy", Locale.ITAL
LocalDate date1 = LocalDate.of(2014, 3, 18);
String formattedDate = date.format(italianFormatter); // 18. marzo 201
LocalDate date2 = LocalDate.parse(formattedDate, italianFormatter);

```

Finally, in case you need even more control, the `DateTimeFormatterBuilder` class lets you define complex formatters step by step by using meaningful methods. In addition, it provides you the ability to have case-insensitive parsing, lenient parsing (allowing the parser to use heuristics to interpret inputs that don't precisely match the specified format), padding, and optional sections of the formatter. You can programmatically build the same `italianFormatter` we used in [listing 12.11](#) through the `DateTimeFormatterBuilder`, for example, as shown in the following listing.

Listing 12.12. Building a `DateTimeFormatter`

```

DateTimeFormatter italianFormatter = new DateTimeFormatterBuilder()
    .appendText(ChronoField.DAY_OF_MONTH)
    .appendLiteral(". ")
    .appendText(ChronoField.MONTH_OF_YEAR)
    .appendLiteral(" ")
    .appendText(ChronoField.YEAR)
    .parseCaseInsensitive()
    .toFormatter(Locale.ITALIAN);

```

So far, you've learned how to create, manipulate, format, and parse both points in time and intervals, but you haven't seen how to deal with subtleties involving dates and time. You may need to deal with different time zones or alternative calendar systems. In the next sections, we explore these topics by using the new Date and Time API.

12.3. Working with different time zones and calendars

None of the classes you've seen so far contain any information about time zones. Dealing with time zones is another important issue that's been vastly simplified by the new Date and Time API. The new `java.time.ZonedDateTime` class is the replacement for the old `java.util.TimeZone` class.

Other formats available



Audiobook



from the complexities related to time zones, Saving Time (DST). Like the other classes of mutable.

12.3.1. Using time zones

A *time zone* is a set of rules corresponding to a region in which the standard time is the same. About 40 time zones are held in instances of the `ZoneRules` class. You can call `getRules()` on a `ZoneId` to obtain the rules for that time zone. A specific `ZoneId` is identified by a region ID, as in this example:

```
ZoneId romeZone = ZoneId.of("Europe/Rome");
```

All the region IDs are in the format "{area}/{city}", and the set of available locations is the one supplied by the Internet Assigned Numbers Authority (IANA) Time Zone Database (see <https://www.iana.org/time-zones>). You can also convert an old `TimeZone` object to a `ZoneId` by using the new method `toZoneId`:

```
ZoneId zoneId = TimeZone.getDefault().toZoneId();
```

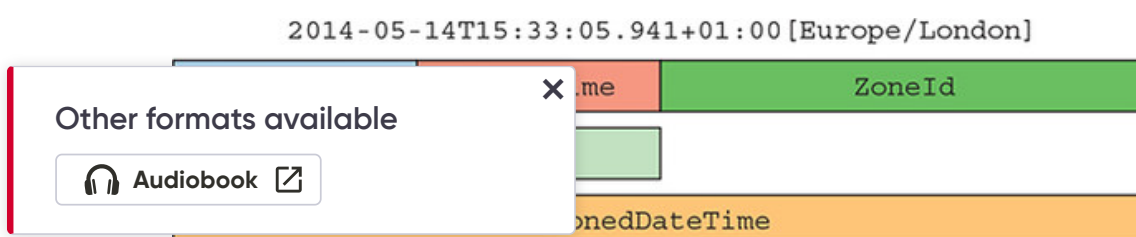
When you have a `ZoneId` object, you can combine it with a `LocalDate`, a `LocalDateTime`, or an `Instant` to transform it into `ZonedDateTime` instances, which represent points in time relative to the specified time zone, as shown in the following listing.

Listing 12.13. Applying a time zone to a point in time

```
LocalDate date = LocalDate.of(2014, Month.MARCH, 18);
ZonedDateTime zdt1 = date.atStartOfDay(romeZone);
LocalDateTime dateTime = LocalDateTime.of(2014, Month.MARCH, 18, 13, 4);
ZonedDateTime zdt2 = dateTime.atZone(romeZone);
Instant instant = Instant.now();
ZonedDateTime zdt3 = instant.atZone(romeZone);
```

Figure 12.1 illustrates the components of a `ZonedDateTime` to help you understand the differences among `LocalDate`, `LocalTime`, `LocalDateTime`, and `ZoneId`.

Figure 12.1. Making sense of a `ZonedDateTime`



You can also convert a `LocalDateTime` to an `Instant` by using a `ZoneId`:

```
LocalDateTime dateTime = LocalDateTime.of(2014, Month.MARCH, 18, 13, 4);
Instant instantFromDateTime = dateTime.toInstant(romeZone);
```

Or you can do it the other way around:

```
Instant instant = Instant.now();
LocalDateTime timeFromInstant = LocalDateTime.ofInstant(instant, romeZone);
```

Note that working with `Instant` is quite useful because you often have to work with legacy code that deals with the `Date` class. There, two methods were added to help inter-operate between the deprecated API and the new Date and Time API: `toInstant()` and the static method `fromInstant()`.

12.3.2. Fixed offset from UTC/Greenwich

Another common way to express a time zone is to use a fixed offset from UTC/Greenwich. You can use this notation to say, “New York is five hours behind London,” for example. In cases like this one, you can use the `ZoneOffset` class, a subclass of `ZoneId` that represents the difference between a time and the zero meridian of Greenwich, London, as follows:

```
ZoneOffset newYorkOffset = ZoneOffset.of("-05:00");
```

The -05:00 offset indeed corresponds to U.S. Eastern Standard Time. Be aware, however, that a `ZoneOffset` defined this way doesn’t have any Daylight Saving Time management, and for this reason, it isn’t suggested in the majority of cases. Because a `ZoneOffset` is also a `ZoneId`, you can use it as shown in [listing 12.13](#) earlier in this chapter. You can also create an `OffsetDateTime`, which represents a date-time with an offset from UTC/Greenwich in the ISO-8601 calendar system:

```
LocalDateTime dateTime = LocalDateTime.of(2014, Month.MARCH, 18, 13, 4);
OffsetDateTime dateTimeInNewYork = OffsetDateTime.of(dateTime.toInstant(romeZone), newYorkOffset);
```

Another advanced feature supported by the new Date and Time API is `ZoneOffset` systems.

Other formats available



Audiobook

Calendar systems

The ISO-8601 calendar system is the de facto world civil calendar system. But four additional calendar systems are provided in Java 8. Each of these calendar systems has a dedicated date class: `ThaiBuddhistDate`,

MinguoDate, JapaneseDate, and HijrahDate. All these classes, together with LocalDate, implement the ChronoLocalDate interface, which is intended to model a date in an arbitrary chronology. You can create an instance of one of these classes out of a LocalDate. More generally, you can create any other Temporal instance by using their from static factory methods as follows:

```
LocalDate date = LocalDate.of(2014, Month.MARCH, 18);
JapaneseDate japaneseDate = JapaneseDate.from(date);
```

Alternatively, you can explicitly create a calendar system for a specific Locale and create an instance of a date for that Locale. In the new Date and Time API, the Chronology interface models a calendar system, and you can obtain an instance of it by using its ofLocale static factory method:

```
Chronology japaneseChronology = Chronology.ofLocale(Locale.JAPAN);
ChronoLocalDate now = japaneseChronology.dateNow();
```

The designers of the Date and Time API advise using LocalDate instead of ChronoLocalDate for most cases, because a developer could make assumptions in his code that unfortunately aren't true in a multicalendar system. Such assumptions might include believing that the value of a day or month will never be higher than 31, that a year contains 12 months, or even that a year has a fixed number of months. For these reasons, we recommend using LocalDate throughout your application, including all storage, manipulation, and interpretation of business rules, whereas you should employ ChronoLocalDate only when you need to localize the input or output of your program.

Islamic calendar

Of the new calendars added to Java 8, the HijrahDate (Islamic calendar) seems to be the most complex because it can have variants. The Hijrah calendar system is based on lunar months. There are a variety of methods to determine a new month, such as a new moon that could be visible anywhere in the world or that must be visible first in Saudi Arabia. The

Other formats available



✕ To choose the desired variant. Java 8 in-
tent for HijrahDate as standard.

The following code illustrates an example of displaying the start and end dates of Ramadan for the current Islamic year in ISO date:

```
HijrahDate ramadanDate =
    HijrahDate.now().with(ChronoField.DAY_OF_MONTH, 1)
        .with(ChronoField.MONTH_OF_YEAR, 9);
System.out.println("Ramadan starts on " +
    IsoChronology.INSTANCE.date(ramadanDate) +
    " and ends on " +
    IsoChronology.INSTANCE.date(
        ramadanDate.with(
            TemporalAdjusters.lastDayOfMonth())));
```

- **1 Get the current Hijrah date; then change it to have the first day of Ramadan, which is the ninth month.**
- **2 IsoChronology.INSTANCE is a static instance of the IsoChronology class.**
- **3 Ramadan 1438 started on 2017-05-26 and ended on 2017-06-24.**

Summary

- The old `java.util.Date` class and all other classes used to model dates and times in Java before Java 8 have many inconsistencies and design flaws, including mutability and some poorly chosen offsets, defaults, and naming.
- All the date-time objects of the new Date and Time API are immutable.
- This new API provides two different time representations to manage the different needs of humans and machines when operating on it.
- You can manipulate date and time objects in both an absolute and relative manner, and the result of these manipulations is always a new instance, leaving the original one unchanged.
- `TemporalAdjusters` allow you to manipulate a date in a more complex way than changing one of its values, and you can define and use your own custom date transformations.
- You can define a formatter to print and parse date-time objects in a specific format. These formatters can be created from a pattern or programmatically, and they're all thread-safe.
- You can represent a time zone, relative to a specific region/location and as a fixed offset from UTC/Greenwich, and apply it to a date-time object to localize it.
- You can use calendar systems different from the ISO-8601 standard

system

Other formats available



Audiobook

